

GenRec: An LLM-Backed Recommendation Ranker at Netflix

Ying Li
yingl@netflix.com
Netflix
Los Gatos, CA, USA

Shradha Sehgal*
shradhasehgal7@gmail.com
Netflix
Los Gatos, CA, USA

Arjun Rao
arjunr@netflix.com
Netflix
Los Gatos, CA, USA

Rein Houthoofd*
rein.houthoofd@gmail.com
Amazon
Sunnyvale, CA, USA

Yaochen Zhu
yzhu@netflix.com
Netflix
Los Gatos, CA, USA

Ashish Rastogi
arastogi@netflix.com
Netflix
Los Gatos, CA, USA

Abstract

Large language models (LLMs) are reshaping recommender systems by enabling richer modeling of users, content, and context directly in natural language. At Netflix, we are exploring this direction through **GenRec**, an LLM-backed recommendation ranker built on top of an in-house foundational LLM. GenRec follows a **two-phase framework**: Phase 1 adapts an open-source LLM to Netflix data, developing deep understanding of the catalog and member behavior while balancing capabilities such as content understanding and instruction following. Phase 2 post-trains this foundation model with recommendation-ranking specific data, labels, and reward signals, aiming to align the ranker with business requirements and long-term member satisfaction.

This paper focuses on Phase 2 and the transition from a traditional discriminative ranker with thousands of engineered features to an LLM-backed ranker driven by verbalized user histories and context. We describe our design for input verbalization and context engineering, post-training data construction, reward integration, model architecture, and a cost-constrained serving design based on prefill-only inference approach. We report results from a large-scale A/B test in comparing GenRec against the current production ranker model, where we show that a GenRec model trained with substantially fewer Phase-2 labeled training examples and input signals can achieve statistically significant gains in offline and online metrics. We discuss how LLM-backed recommenders could shift the recommendation paradigm: from feature engineering to context engineering, and from bespoke architectures to shared foundation backbones. We also outline practical lessons for serving such systems under real-world resource constraints.

1 Introduction

The recommendation system is central to the experiences of hundreds of millions of members at Netflix. Our current recommender systems in production generate personalized title recommendations based on large number of hand-crafted features spanning user, item, and interaction signals, along with extensive higher-order feature interactions, in line with prior large-scale recommendation systems [3, 4, 11, 15]. They also employ customized, complex architectures tailored to different recommendation needs. For example, specialized networks for modeling feature interactions, transformer-based architectures for capturing dynamic member

interests, and multi-task learning architectures powering multiple recommendation tasks [10, 14, 27, 28].

These models have iterated over many years to generate recommendations for a variety of content types (movies, series, games, live events, podcasts, etc.) across the Netflix product. While this architecture has become a critical part of our member experience, it has become increasingly difficult to support new business needs. For example, onboarding a new content type or recommendation surface often requires substantial work in feature engineering, model architecture design, infrastructure changes, and experimentation. As a result, the system’s complexity makes it difficult to keep pace with the rapid growth of business needs at Netflix.

Recently, large language models (LLMs) are beginning to reshape how recommendation systems are designed, deployed, and leveraged across the industry [9, 13, 23]. Their broad world knowledge and expressive language understanding enable richer modeling of users, content, and context than traditional architectures. This opens up new possibilities of the next generation of recommendations that represent user histories and contextual signals directly as natural language to generate better recommendations with deeper understanding of user preferences and item semantics. It also enables quicker adaptations to new use cases, such as steering recommendations via prompts rather than re-designing new specialized models and pipelines.

However, out-of-the box LLMs are not yet suitable as production-level recommenders out of the box: they tend to over-recommend globally popular content, hallucinate out-of-catalog titles, ignore nuanced business constraints, and offer limited personalization. To address these issues, we built **GenRec**, an LLM-backed recommendation ranker that post-trains an internal foundation LLM on Netflix corpora. Specifically, we focus on a full-catalog ranking use case, and demonstrate that a LLM-based recommendation ranker is a viable and promising path. GenRec represents **our first step** toward transitioning from a traditional recommendation stack to an LLM-native system.

To generate recommendations with GenRec, we first verbalize user interaction histories, item metadata, and context information as natural language, then tokenize this textual representation and feed it into the LLM (Figure 1). To learn Netflix-specific user preferences and avoid over-recommending popular items, we post-train the model on Netflix’s catalog and member behavior data. We align the recommendation outputs with long-term member satisfaction and business objectives by incorporating multiple reward signals during training. We further make the model “Netflix-catalog-aware”

*Work done while employed at Netflix.

by constraining its outputs to the items in the Netflix catalog, which eliminates out-of-catalog recommendations. Finally, we optimize GenRec under cost constraints and integrate it with Netflix’s LLM serving stack.

Compared to the existing production recommendation ranker, GenRec substantially simplifies the feature engineering. Instead of manually engineering complex feature interactions, we rely on the LLM model to infer these interactions and user preferences directly from the raw input text. This shifts the recommendation problem from **feature engineering** to **context engineering**: deciding what information to present to the model, how to express it within a limited token budget, and what objective and rewards to optimize for, rather than iteratively designing and integrating new features and feature interactions.

GenRec also benefits from the capabilities of the pre-trained foundation LLM—such as user understanding and content understanding—so we do not need to build these capabilities from scratch. As a result, we observe that GenRec can reach parity with a mature production ranker using substantially fewer Phase-2 labeled examples and input signals than the traditional models. This marginal data efficiency at the Phase-2 post-training stage is especially valuable because Phase-1 foundation training runs at a much lower cadence, whereas Phase-2 models are refreshed more frequently.

We evaluate GenRec using both offline experiments and a large-scale online A/B test against a mature production baseline. In the configuration tested, GenRec uses only a small fraction of the Phase-2 labeled training data and input signals of the current production stack, yet achieves statistically significant improvements on both offline and online metrics, while reducing reliance on hand-engineered features and bespoke architectures. Together, these results suggest that LLM-backed rankers are a viable and promising alternative to traditional models, at least on the recommendation surfaces tested. The contributions of this paper include:

- **LLM-backed ranking that is competitive with a mature production system.** We present GenRec, an LLM-backed recommendation ranker built on top of an in-house foundation LLM, and compare it to a long-running, highly engineered production ranker. A GenRec model trained with substantially fewer Phase-2 training examples and input signals achieves statistically significant gains on our primary offline and online metrics.
- **A two-phase, LLM-centric training framework with quantified benefits.** We validate a two-phase training scheme in which Phase 1 adapts an open-source LLM to Netflix data to build a Netflix-aware foundation model, and Phase 2 performs high-cadence post-training for recommendation ranking. We articulate the distinct roles and objectives of the two phases and empirically decompose their impact on ranking quality, showing substantial gains from both phases.
- **A LLM-backed ranking architecture with catalog-aware scoring.** We describe an architecture that uses a generative, decoder-only LLM backbone, and a catalog-aware scoring head that scores large item catalogs in a single forward

pass. This design enables large candidate-set ranking while restricting recommendations to in-catalog items.

- **Context engineering and reward-weighted training as practical levers for quality–cost control.** We treat context engineering and reward integration as first-class design dimensions. On the context side, we show that careful verbalization design can reduce the effective context length to about a **third** of the original token budget with negligible loss in offline ranking quality, yielding a similar factor reduction in serving cost. On the objective side, we integrate multiple reward signals via a reward-weighted ranking loss, steering the model toward long-term value and business goals in a robust and cost-efficient way.
- **Empirical observation on data/model scaling and quality–cost trade-offs.** We characterize how recommendation quality scales with Phase-2 post-training data and with backbone sizes, and how these gains interact with training and serving costs. We observe consistent, monotonic data scaling behavior across model sizes, and identify practical “sweet spots” on the quality–cost Pareto frontier for Phase-2 configurations. These findings offer actionable guidance for designing LLM-backed recommenders under resource constraints.

The rest of the paper is organized as follows. Section 2 situates our work within existing research on generative and LLM-based recommendation. Section 3 formalizes the problem setting. Section 4 describes our methodology, including input verbalization, model architecture, post-training data and rewards, and serving optimizations. Section 5 presents results from offline experiments and a large-scale online A/B test. Section 6 discusses potential shifts introduced by LLM-native recommendation. Section 7 concludes and outlines directions for future work.

2 Related Work

2.1 Generative Recommendations with Special Tokens

A growing line of work models recommendation as autoregressive generation over user interaction sequences. Sequential recommenders such as SASRec [10] and BERT4Rec [21] established this template with next-item prediction over sequences of item IDs. Industrial systems have since scaled this paradigm using transformer-decoder backbones whose vocabularies are extended with special tokens encoding items ID, item metadata, locale, time, devices, surfaces, and so on, allowing the model to condition on heterogeneous information [1, 16, 25].

A complementary direction tokenizes items into semantic codes to capture the semantic meaning of items. TIGER [19], for example, represents each item as a short sequence of “semantic IDs” learned via an RQ-VAE over content embeddings, enabling autoregressive decoding over billion-item catalogs. Variants of this approach have been adopted across multiple platforms, including for ranking at Google [20] and end-to-end retrieval-and-ranking at Kuaishou [6].

These models have demonstrated strong performance, but they are typically limited in their ability to leverage broad world knowledge, to steer recommendations through flexible prompts, or to

¹The verbalization example shown is illustrative only and does not reflect actual user data in production.

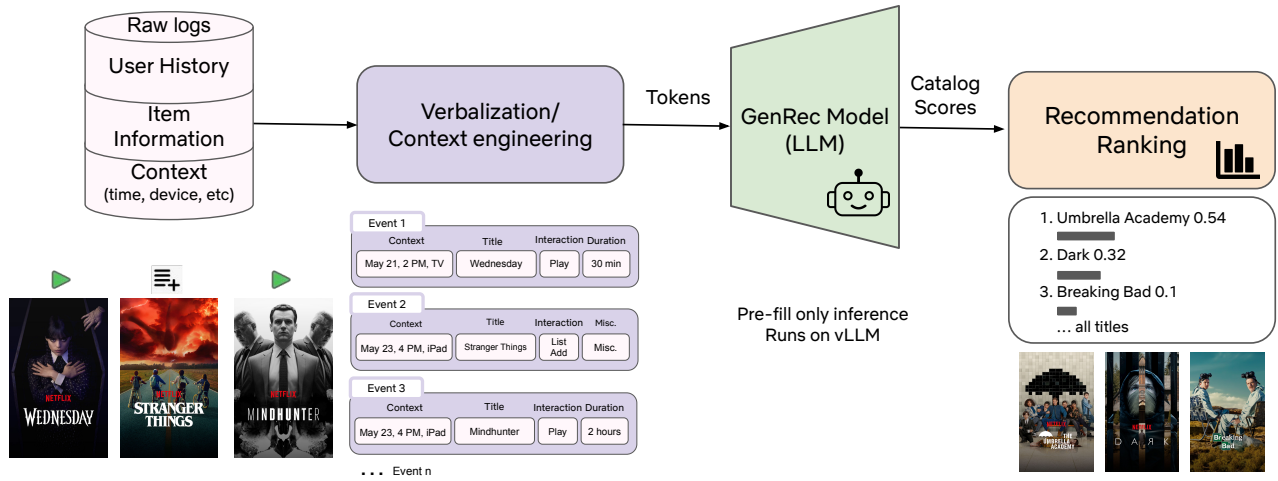


Figure 1: GenRec inference pipeline. Raw logs of user history, item metadata, and context are transformed via context engineering into natural-language prompts and fed into the GenRec model, which runs on vLLM in prefill-only mode and outputs scores for the entire item catalog, inducing a recommendation ranking.¹

inherit the richer capabilities of modern pretrained language models. These limitations have motivated a parallel line of work that builds recommenders directly on top of LLM backbones.

2.2 LLM-based Recommendation

The success of large language models in various tasks has inspired a line of work that uses pretrained LLMs as the backbone of recommenders, rather than training recsys-specific transformers from scratch with special tokens. Early academic systems cast recommendation as text-to-text generation on top of language models [2, 7, 24].

To allow LLMs to generate items directly rather than free-form text, recent industrial systems extend the LLM vocabulary with Semantic-ID item tokens. Google’s PLUM [8] adapts a pretrained LLM at YouTube scale through Semantic-ID tokenization, continued pre-training on domain-specific data, and task-specific fine-tuning. Spotify’s GLIDE [5] formulates podcast discovery as instruction-following over a Semantic-ID catalog. Kuaishou’s OneRec-Think [12] extends OneRec with a Qwen3 model backbone and reasoning steps.

Most LLM-based generative retrieval systems rely on autoregressive decoding with beam search at inference time, which introduces latency overhead that can be prohibitive at scale, especially for large candidate sets. Our work avoids these limitations by augmenting the base LLM architecture with a catalog-aware ranking head, so that it can rank a large candidate set in a single forward pass, which makes the serving more cost-effective. In addition, we design the system explicitly through an LLM lens, including effective context engineering and reward integration to satisfy business requirements and optimize long-term user satisfaction within real-world resource constraints.

3 Problem Setting

In GenRec, we focus on a full-catalog ranking problem (or top K ranking if candidate set is provided). The resulting ranked list serves

as a personalized representation of user preferences that can be reused across diverse use cases, including item ranking on various surfaces and personalized input signals to downstream applications.

Notation. Let U denote the set of users, C the catalog of items (movies, shows, games, live events, podcasts, etc.), and X the space of contexts (device, surface, locale, time of day, etc.). Here, we consider a single recommendation request characterized by a user $u \in U$, a context $\tau \in X$, and a time t ; let H denote the user’s interaction history prior to t . Each item $i \in C$ is associated with metadata M_i (title name, genre, synopsis, release date, etc.).

Ranking Target. A recommender maps a request (u, τ, t, H) to a ranking π of the catalog C , $\pi : C \rightarrow \{1, \dots, |C|\}$, where $\pi(i)$ denotes the position assigned to item i (with rank 1 at the top of the list). The objective is to select a ranking π that maximizes expected long-term member utility — a proxy for member satisfaction and retention — rather than short-term engagement alone.

4 Methodology

4.1 Overview

GenRec follows a **two-phase training framework** (Figure 2). In **Phase 1**, a foundational LLM is trained on top of OSS models with proprietary Netflix data to develop deep understanding of Netflix user and content. In **Phase 2**, we further post-train this foundational model on ranking-application specific data and objectives, producing GenRec, which is tailored and optimized for the recommendation ranking task.

Compared to Phase 1, GenRec Phase-2 post-training pursues a few different objectives:

- (1) **Capability focus:** Phase 1 optimizes and balances a broad set of foundational capabilities, such as world knowledge, personalization capability, content understanding, and language capability. Phase 2, by contrast, focuses more on

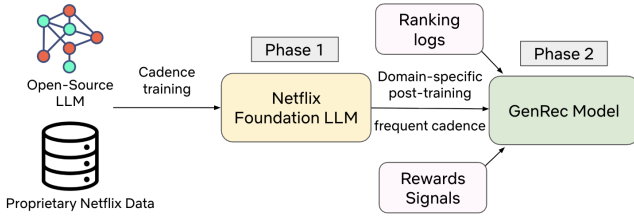


Figure 2: Two-phase training framework. Phase 1 trains a foundational LLM on Netflix data for user and content understanding, and Phase 2 post-trains on recommendation-ranking-specific data and objectives.

recommendation ranking capabilities, such as ranking quality and recommendation steering.

- (2) **Update cadence:** Phase 1 is updated relatively infrequently, targeting long-term user and content understanding and language capabilities. Phase 2 needs to be updated more frequently to track new content launches, evolving popularity patterns, and members’ latest interests.
- (3) **Cost sensitivity:** Phase 1 aims to build the most capable foundational model and is less constrained by serving cost; while Phase 2 must be explicitly cost-efficient to serve large-scale traffic.

4.2 Post-Training Data

At Netflix, members generate **hundreds of billions** of interaction events across diverse content types, including movies, shows, games, lives, and more. These interactions span a wide range of engagement signals — viewing, playing, thumbing, adding to list, etc. — and occur across multiple recommendation surfaces.

To post-train LLMs with recommendation capabilities, we transform a member’s historical interaction data into a single-turn or multi-turn “conversation” between a user and a recommender. Each turn pairs a **user message** (combining context, history, and a task) with an **assistant message** representing the member’s actual engagement for that turn. Specifically, each conversation contains:

- **Context:** surface, time, device, locale, etc.
- **User profile & history:** country, tenure, plan, historical interactions, etc.
- **Item-level detail:** item IDs, item metadata (title name, release time, synopsis, etc.), popularity trends, etc.
- **Task:** for example, predict the next items that the user will play or thumb up
- **Assistant message:** the user’s implicit or explicit feedback (play, play duration, abandon, thumb, etc.), which acts as the ground-truth signal during training.

During Phase-2 post-training, the LLM is optimized on this conversational data to model how assistant messages depend on the preceding user messages. This conversational framing provides a unified way to express rich recommendation logs as text, which is then used to train both the language modeling objective and the catalog-aware ranking objective described in the Model Architecture section.

4.3 Input Verbalization and Context Engineering

Unlike traditional recommenders that encode user interactions and item metadata into hand-crafted features or dense embeddings, GenRec **verbalizes** rich user interaction histories and context as natural language or lightly structured text. By encoding raw interaction signals directly in the LLM’s semantic space, GenRec relies on the LLM to learn higher-level patterns—such as item relationships, temporal dynamics, and evolving user interests—rather than encoding these relationships explicitly through manual feature engineering.

Verbalization is not merely converting signals into text; it requires deliberate **context engineering** [18] to accommodate long user context under the finite token budget. A member’s interaction history can easily exceed the token budget, both because of the large number of interactions and the richness of per-interaction metadata. The context window is a limited and valuable resource, where excessively long contexts can dilute attention and significantly increase training and inference costs.

We therefore carefully curate which events and attributes enter the context window:

- **Retain in full:** high-signal engagements, such as long-duration plays and thumbs-up events, are verbalized with richer metadata.
- **Omit entirely:** recent, low-signal events—such as very short plays or noisy views/clicks—are omitted, as they contribute little to ranking quality relative to their token cost.
- **Summarize or compress:** repetitive behaviors, such as binge-watching sessions, are compressed instead of listing every event repeatedly.
- **Elaborate selectively:** for important items (e.g. new releases or cold-start items), we can include more detailed metadata to compensate for the limited information available from the pretrained model and historical interactions.

Within the fixed token budget, we prioritize short- to medium-term history with higher granularity, while older history is either omitted or compressed into a brief user-interest summary. The overall goal is to construct a compact set of highly informative tokens that preserves recommendation quality without exceeding the context window or incurring prohibitive inference costs.

4.4 Post-Training Objective

GenRec Phase-2 training combines mainly two main objectives:

- (1) **Recommendation ranking objective.** The primary task is a catalog-aware ranking objective that trains the model to assign higher scores to items that correspond to high-value engagements. In this task, positive labels correspond to high-quality engaged events (e.g., high-duration plays, strong explicit feedback), with denoising logic and thresholds that vary by content types. These labels are used in a cross entropy loss over the item catalog (or candidate set), teaching the model to produce a high-quality ordering of items given the verbalized context.
- (2) **Language modeling objective.** In addition, we include a language modeling objective over the verbalized inputs and outputs (such as predicted titles and other textual fields).

This task helps maintain the backbone’s general language understanding and text generation capabilities, which are important for (i) modeling rich, natural-language user histories and item metadata, and (ii) enabling recommendation steering via prompts.

During **training time**, the model is optimized jointly to (i) score items for ranking and (ii) model the textual domain (e.g., predicting tokens in titles and verbalized context). During **inference time**, we currently use only the ranking task to generate recommendation ranking. The language modeling objective primarily serves to introduce complementary textual information to improve the ranking task, preserve language understanding and responsiveness to prompt-based steering, and keep open the possibility of future text generation use cases (e.g., natural-language explanations).

The overall GenRec model is trained with a multi-objective loss that combines the recommendation ranking objective, the language modeling objectives, and other objectives where applicable.

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{ranking}} + \beta \cdot \mathcal{L}_{\text{language}} + \gamma \cdot \mathcal{L}_{\text{miscellaneous}}$$

$$\text{s.t. } \alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0$$

The α, β, γ are tunable hyperparameters which can be tuned through offline experiments.

4.5 Model Architecture

GenRec’s backbone architecture closely follows the foundational LLM: a **decoder-only Transformer** trained with next-token-prediction style objectives. It is augmented with a **catalog-aware ranking head** that constrains recommendations to the Netflix in-catalog items and supports efficient scoring over large candidate sets.

Scoring Model. The scores are computed in the following three stages:

- (1) **Verbalization.** A verbalizer V maps the interaction history, the context, and the item metadata into a single text sequence:

$$x = V(H, \{M_i\}_{i \in C}, \tau) \quad (1)$$

- (2) **Pooled representation.** The LLM encodes x into a d -dimension representation h , taken as the hidden state at a pooling position, which summarizes the user preference and context.

- (3) **Catalog-aware scoring.** A scoring head ϕ assigns each item i a score by combining h with a learned d -dimensional embedding e_i as the recommendation ranking task.

The parameters θ , including the weights of LLM, the scoring head ϕ , and the item embeddings $\{e_i\}$, are trained jointly. A softmax over the catalog converts these scores into a probability distribution. Afterwards, the catalog ranking π can be derived from the item scores. When the catalog is too large to score exhaustively, we can combine the scoring with sampled-softmax instead.

4.6 Reward Signals

Beyond raw recommendation accuracy, GenRec must also satisfy two additional goals:

- (1) **Respect business requirements**, such as content mixing logic (e.g., balance movies, shows, live, games, podcast, and other content types).

- (2) **Maximize long-term member satisfaction**, rather than just optimizing short-term user engagements.

Simply post-training on raw interaction sequences risks drifting away from these goals. For example, the model might over-recommend binge-watching over discovery, favor videos over games, or select items that maximize immediate clicks at the expense of catalog exploration or long-term retention. To address this, we incorporate reward signals derived from separate reward models into GenRec’s training objective. We use two broad categories of rewards:

- (1) **Long-term satisfaction proxies.** These rewards estimate how strongly a short-term engagement event correlates with long-term outcomes. Because true long-term metrics are noisy and delayed, we rely on more stable proxies learned from historical data. For example, the reward models assign higher values to engagements that encourage members to return to the service, explore across a wider portion of the catalog, or lead to sustained engagement over time.
- (2) **Behavior rebalancing across content and engagement types.** Additional rewards are used to rebalance behavior across content types (e.g., movies vs. games vs. live vs. podcast) and launch stage (e.g. pre-launch, newly launched, ever-green), in order to meet business objectives such as exposure fairness across content types.

We leverage multiple reward models from an existing reward modeling framework [22] and integrate their outputs into GenRec via a **reward-weighted ranking loss**. Concretely, each training example is assigned a scalar weight derived from the multiple reward signals, and the ranking loss for that example is scaled accordingly. High-value engagements (according to the reward models) receive larger weights, while low-value or undesirable behaviors are down-weighted.

This reward-weighted loss provides a simple, robust, and cost-effective way to steer the model toward long-term value and business-aligned behavior. Compared to full reinforcement learning approaches, this approach is easier to deploy and maintain. In preliminary experiments, RL-style methods (e.g. GRPO) showed additional gains over supervised finetuning, but their high training overhead makes them better suited for future work. Therefore, currently we adopt the reward-weighted loss approach as the main alignment mechanism for GenRec, due to its simplicity, stability, and cost considerations.

4.7 Online Serving and Cost Optimizations

GenRec is served with Netflix’s internal LLM serving stack using the vLLM framework [17]. At Netflix scale (hundreds of millions of members globally), serving cost is a first-class constraint, and inference cost in practice is roughly proportional to the product of model size and context length. We therefore pursue two complementary strategies to optimize the serving cost. First, we **explore smaller and distilled language models** [26] trained on larger or more targeted datasets, aiming to recover much of the quality of larger models while reducing per-request computation. Second, we **aggressively optimize context length via verbalization compaction**, as described in the previous section, reducing the tokens used to represent input signals without significantly degrading offline ranking quality.

A further lever on inference cost is the **inference mode**. Since GenRec needs to rank items over a large candidate set, naïve autoregressive decoding (e.g. generating one token at a time or using beam search) would be prohibitively expensive. Although the underlying backbone is a generative, decoder-only LLM capable of autoregressive decoding, we intentionally deploy GenRec in a **prefill-only** configuration: the model consumes the input context once, and produces ranks for the full candidate set in a single forward pass. This avoids step-by-step decoding and makes it feasible to serve GenRec on high-volume workloads within our compute budget.

5 Experiments & Results

In this section we evaluate GenRec against the production model, analyze how performance scales with data and model size, quantify the contributions of Phase-1 and Phase-2 training, and study the impact of context length optimization.

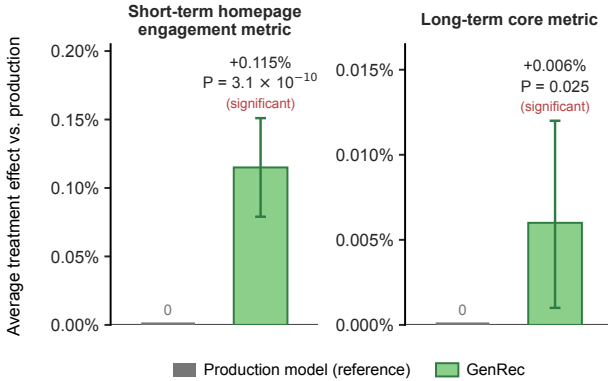


Figure 3: Online metrics of GenRec vs. production model. GenRec achieves statistically significant improvements on both short-term and long-term online metrics.²

5.1 Comparison of GenRec vs. Production Baseline

We compared GenRec with the production baseline, a mature discriminative model tuned over many years.

Offline, GenRec exceeds the production model on ranking metrics, Mean Reciprocal Rank (MRR), while using significantly less Phase-2 training data and fewer input signals. Concretely, with about 40× less Phase-2 labeled training examples than the production model, GenRec achieves an offline lift of approximately +1.6% relative in MRR over the baseline. Moreover, offline metrics continue to improve as we further scale up the training data and input signals.

We validate this result in a large-scale online A/B test, focusing on key batch-compute surfaces. We allocated approximately 10% of Netflix traffic for 4 weeks. Under a low-data, low-signals configuration, the Phase-2 GenRec model achieves statistically significant improvement over the production baseline on both short-term and long-term online metrics (see Figure 3). For example, we observe

a +0.006% relative improvement on our core online metric, which is statistically meaningful at Netflix scale. These findings demonstrate that an LLM-backed ranker, when combined with appropriate post-training and reward signals, can become a viable and promising alternative to traditional recommendation ranker.

The relatively small number of training samples and input signals used by GenRec, compared to the production ranker, illustrates the marginal data efficiency of Phase-2 post-training. Since Phase 2 is updated much more frequently than Phase 1, improving data efficiency at this stage is especially valuable in practice, as it directly reduces the compute required to keep the ranker fresh.

5.2 Data and Model Scaling

Data Scaling: We first study how recommendation quality scales with the volume of Phase-2 post-training data. Holding other variables fixed, we train GenRec on datasets ranging from 1x to 20x the size of our smallest data size configuration. We run this study for two backbone sizes: a smaller model (on the order of ~1B parameters) and a larger model (on the order of ~10B parameters). For both model sizes, we observe that offline ranking metric MRR improves monotonically as we increase the amount of Phase-2 data (Figure 4). Although the absolute MRR of the ~1B model is lower than that of the ~10B model, it exhibits a similar scaling pattern to ~10B model.

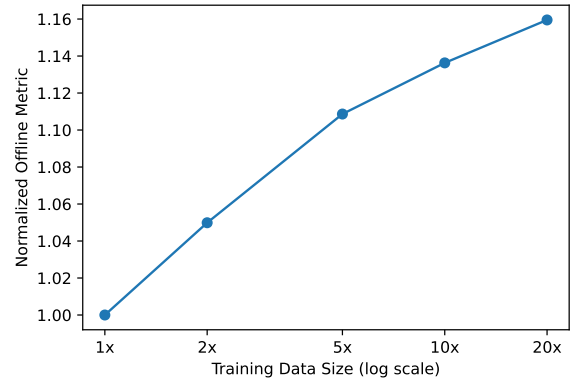


Figure 4: Phase-2 data scaling for the ~10B model.

Model Scaling: We also examine how recommendation quality varies with backbone size. We post-train a few GenRec variants with base models of different sizes (on the order of ~1B to ~10B parameters) under a fixed training budget (same GPU configuration and similar wall-clock training time). Under this constraint, larger backbones consistently achieve higher offline MRR than smaller ones.

Quality-Cost Tradeoff in Phase 2: While data and model scaling both improve quality, they also increase **training and serving costs**, especially for GenRec that is trained at a higher cadence and directly serves large-scale online traffic. We therefore explicitly search for Phase-2 configurations that balance quality and cost: large enough to capture most of the incremental quality gains, but not so large that post-training or serving becomes prohibitively expensive. In practice, we use the data- and model-scaling curves,

Table 1: Impact of Phase 1 vs. Phase 2 training

	Offline ranking metrics (MRR)
Impact of Phase 1	+ 10-20% compared to using an OSS model as base
Impact of Phase 2	+ 35-50% compared to a freshly trained Phase 1, with the gain increasing over time

together with context-length ablations, to identify a “sweet spot” on the quality–cost Pareto frontier that recovers most of the achievable quality improvement while staying within a given compute budget.

5.3 Impact of Phase 1 & Phase 2 on Recommendation Task

We quantify the contributions of Phase-1 foundation training and Phase-2 post-training on offline ranking metrics.

Phase 1. Using the Phase-1 foundational LLM as the base model improves offline ranking metrics by the order of **10-20%** compared with using an off-the-shelf LLM as the backbone. This underscores the importance of the personalization and content-understanding capabilities inherited from the Phase 1 foundational model.

Phase 2. Phase-2 post-training provides a further roughly **35-50%** gain in offline ranking metrics when evaluated on the Phase-1 training cutoff date (i.e. when Phase-1 model is the freshest). This highlights the value of task-specific adaptation, including domain-specific data, rewards, and verbalizations. In addition, the relative benefit of Phase 2 grows over time: the improvement rises to roughly 80% after two weeks, which is expected. This growing gain reflects both the strength of task-specific adaptation and the increasing staleness of the less frequently updated Phase-1 backbone, for example with respect to shifting content popularity trends and evolving member interests.

5.4 Context Length Optimization

The context window in an LLM is both a quality driver and a cost driver: longer verbalizations expose more information about user behavior and context but increase training and serving cost. We therefore study how context length and verbosity affect recommendation performance and explicitly search for compact verbalizations that preserve quality. We build on the “Input Verbalization and Context Engineering” heuristics in a few steps.

- **First, we perform event selection and compression.** We omit low-signal or noisy engagements, compress repetitive activity, and retain high-signal interactions. This yields a cleaned, ordered sequence of engagements that are candidates for inclusion in the context.
- **Second, we search for an optimal history length to elaborate.** Starting from this cleaned sequence, we vary the number of historical events we keep in the prompt. Concretely, we sweep the number of retained engagements, and plot MRR against the number of engagement events. This

reveals an elbow point: quality improves as we increase context length up to that point, after which additional tokens yield only marginal gains (Figure 5).

- **Finally, we construct a concise and efficient verbalization.** For each retained event, we construct prompts with different levels of detail (e.g. different engagement details, simplifying wording, and removing few-shot examples) and measure offline performance.

In our experiments, we observe the context length can be reduced to roughly **a third** of the original token budget (for example, from about 5,000 tokens down to about 1,700 tokens) with only negligible degradation in offline ranking metrics. This corresponds to a substantial reduction in context size—and therefore inference cost—while maintaining essentially the same model quality. Since GenRec is largely compute-bound with serving cost approximately proportional to the context length, we observed the serving cost is reduced to roughly **a third** of original cost as well.

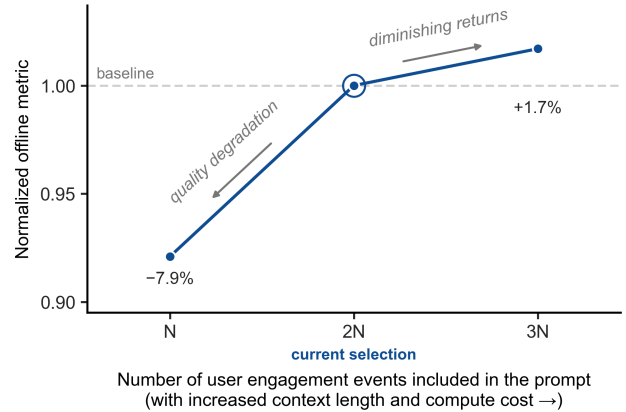


Figure 5: Normalized offline ranking metric (MRR) vs. number of user engagement events included in the prompt. The dashed line marks the baseline elbow point: increasing the number of events beyond this yields diminishing returns.

6 Discussion: LLM-Native Recommendation

GenRec is an initial step toward a more LLM-centric recommendation stack at Netflix. Moving from traditional recommendation systems to LLM-based ones is not just “swapping the ranker with a transformer”: it changes how we represent users and content, how we structure models and training, and how we design serving infrastructure. Below, we outline several of these changes and how they manifest in GenRec. Taken together, they move our recommendation stack closer to the broader LLM paradigm.

6.1 From Feature Engineering to Context Engineering

Traditional systems are built around large number of handcrafted features, supported by substantial infrastructure for aggregation, freshness, interaction design, and leakage control. In contrast, LLM-centric

systems prioritize constructing rich textual contexts from raw interaction sequences, content metadata, context, tool outputs, or user memory. The “prompt” becomes the new feature vector. Modeling effort shifts from designing individual features to context engineering: deciding which signals to include, how to summarize them, how far back in time to go, and how to encode them within a finite token budget. Our experiments on verbalization compaction illustrate this shift: careful context design can preserve quality while dramatically reducing serving cost.

6.2 From Customized Architectures to Foundation Backbones

Historically, recommender systems have relied on a wide variety of bespoke architectures—two-tower models, DLRM-style feature interaction networks, custom attention blocks, and large multi-task setups. LLM-centric systems instead standardize on transformer backbones inherited from a foundation model. Innovation moves up a level: from per-task architecture design to questions of data, scaling laws, post-training strategy, and inference optimization. In GenRec, we leverage the same LLM backbone as the foundational LLM, rather than designing a new architecture from scratch. The LLM backbone also enables flexible recommendation steering through natural-language inputs to support new member experiences.

6.3 Scaling Laws as a Design Guide

Classical RecSys stacks can hit diminishing returns due to sparse IDs, heavily engineered objectives, and task-specific architectures. In an LLM-backed setting, the recommendation model shares its backbone with a pre-trained LLM and inherits its data and model scaling behavior. Our studies show clear scaling trends in both data and model size: performance improves monotonically as we add more Phase-2 training data, and larger models consistently outperform smaller ones under fixed training budgets. This brings RecSys closer to the broader LLM paradigm, where scaling laws are treated as a central design guide rather than an after-the-fact observation.

6.4 Pre-Training, Post-Training, and Reward Alignment

Instead of building each recommender from scratch, an LLM-centric approach starts from a pre-trained foundation model and then uses task-specific post-training and reward alignment to adapt this shared backbone. In GenRec, Phase 1 provides a Netflix-aware foundation LLM, and Phase 2 performs lightweight, recommendation-specific adaptation with labels, rewards, and verbalizations. This pattern makes it easier to scale innovation across applications: multiple use cases can reuse the same foundation and infrastructure, rather than re-implementing similar capabilities in separate, siloed systems.

6.5 From RecSys Infrastructure to LLM Infrastructure

LLM-backed recommenders must process longer user-history contexts, which makes serving efficiency a central design concern. At the same time, LLMs are inherently expensive to train and serve,

so cost optimization must shape choices around model size, context length, and decoding strategy. Techniques such as KV-caching, prefix caching, and prefill-only inference become key levers for keeping inference viable within cost budgets. As a result, the online serving stack for recommendation increasingly resembles modern LLM infrastructure—GPU-accelerated, vLLM/Triton-based, with careful batching and caching—rather than classic RecSys infrastructure built around MLPs or factorization models.

7 Conclusions & Future Work

We presented GenRec, an LLM-backed recommendation ranker at Netflix, and described how we adapt an in-house foundational LLM into a ranker suitable for production traffic on batch-compute surfaces. By verbalizing user histories and context, adding a catalog-aware ranking head, and training with multiple reward signals, we obtain an LLM-backed recommender that, despite using far less Phase2-2 labels and explicit input signals than our long-standing production ranker, achieves statistically significant improvement over it on both short-term and long-term online metrics in the settings we study.

Overall, GenRec represents an initial step toward a more LLM-centric recommendation stack at Netflix. Our results suggest that this is a viable and promising path for large-scale personalization in at least some scenarios, provided we continue to manage cost and infrastructure complexity carefully and to evaluate quality trade-offs rigorously against strong non-LLM baselines.

8 Acknowledgments

GenRec is the result of close collaboration among multiple teams and organizations across Netflix. The contributors to this work (in alphabetical order):

- **AI for members:** Arjun Rao, Ashish Rastogi, Baolin Li, Fernando Amat Gil, Grace Huang, Justin Basilico, Kamelia Aryafar, Linas Baltrunas, Moumita Bhattacharya, Oghe-neovo Dibie, Rein Houthooft, Shradha Sehgal, Sejoon Oh, Sergi Perez, Sourabh Medapati, Thea Wang, Yaochen Zhu, Yesu Feng, Ying Li, Yun Li, Yucheng Shi, Yunan Hu
- **AI platform and serving:** Abhishek Agrawal, Adam Singer, Binh Tang, Daneo Zhang, Derek Olejnik, Ed Maddox, Erik Osheim, Lingyi Liu, Liping Peng, Meghana Chilukuri, Nicolas Hortiguera, Shaojing Li, ZQ Zhang
- **Product:** Ilke Kaya, Michelle Kislak, Scarlet Chen, Si Cheng

References

- [1] Prabhat Agarwal, Anirudhan Badrinath, Laksh Bhasin, Jaewon Yang, Edoardo Botta, Jiajing Xu, and Charles Rosenberg. 2026. PinRec: Outcome-Conditioned, Multi-Token Generative Retrieval for Industry-Scale Recommendation Systems. arXiv:2504.10507 [cs.LR] <https://arxiv.org/abs/2504.10507>
- [2] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems* (Singapore, Singapore) (RecSys '23). Association for Computing Machinery, New York, NY, USA, 1007–1014. doi:10.1145/3604915.3608857
- [3] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & Deep Learning for Recommender Systems (DLRS 2016). Association for Computing Machinery, New York, NY, USA, 7–10. doi:10.1145/2988450.2988454

- [4] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems* (Boston, Massachusetts, USA) (*RecSys '16*). Association for Computing Machinery, New York, NY, USA, 191–198. doi:10.1145/2959100.2959190
- [5] Edoardo D’Amico, Marco De Nadai, Praveen Chandar, Divita Vohra, Shawn Lin, Max Lefarov, Paul Giglioli, Gustavo Penha, Ilya Kopytsky, Ivo Joel Senese, Darren Mei, Francesco Fabbri, Oguz Semerci, Yu Zhao, Vincent Tang, Brian St. Thomas, Alexandra Ranieri, Matthew N. K. Smith, Aaron Bernkopf, Bryan Leung, Ghazal Fazelnia, Mark VanMiddlesworth, Timothy Christopher Heath, Petter Pehrson Skiden, Alice Y. Wang, Doug J. Cole, Andreas Damianou, Maya Hristakeva, Reid Wilbur, Tarun Chillara, Vladan Radosavljevic, Pooja Chitkara, Sainath Adapa, Juan Elenter, Bernd Huber, Jacqueline Wood, Saaketh Vedantam, Jan Stypka, Sandeep Ghael, Martin D. Gould, David Murgatroyd, Yves Raimond, Mounia Lalmas, and Paul N. Bennett. 2026. Deploying Semantic ID-based Generative Retrieval for Large-Scale Podcast Discovery at Spotify. arXiv:2603.17540 [cs.IR] <https://arxiv.org/abs/2603.17540>
- [6] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. arXiv:2502.18965 [cs.IR] <https://arxiv.org/abs/2502.18965>
- [7] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5). In *Proceedings of the 16th ACM Conference on Recommender Systems* (Seattle, WA, USA) (*RecSys '22*). Association for Computing Machinery, New York, NY, USA, 299–315. doi:10.1145/3523227.3546767
- [8] Ruining He, Lukas Heldt, Lichan Hong, Raghunandan Keshavan, Shifan Mao, Nikhil Mehta, Zhengyang Su, Alicia Tsai, Yueqi Wang, Shao-Chuan Wang, Xinyang Yi, Lexi Baugher, Baykal Cakici, Ed Chi, Cristos Goodrow, Ningren Han, He Ma, Romer Rosales, Abby Van Soest, Devansh Tandon, Su-Lin Wu, Weilong Yang, and Yilin Zheng. 2026. PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations. In *Proceedings of the ACM Web Conference 2026* (United Arab Emirates) (*WWW '26*). Association for Computing Machinery, New York, NY, USA, 8093–8104. doi:10.1145/3774904.3792802
- [9] Yupeng Hou, An Zhang, Leheng Sheng, Zhengyi Yang, Xiang Wang, Tat-Seng Chua, and Julian McAuley. 2025. Generative Recommendation Models: Progress and Directions. In *Companion Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) (*WWW '25*). Association for Computing Machinery, New York, NY, USA, 13–16. doi:10.1145/3701716.3715856
- [10] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. arXiv:1808.09781 [cs.IR] <https://arxiv.org/abs/1808.09781>
- [11] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37. doi:10.1109/MC.2009.263
- [12] ZhanYu Liu, Shiyao Wang, Xingmei Wang, Rongzhou Zhang, Jiaxin Deng, Honghui Bao, Jinghao Zhang, Wuchao Li, Pengfei Zheng, Xiangyu Wu, Yifei Hu, Qigen Hu, Xinchun Luo, Lejian Ren, Zixing Zhang, Qianqian Wang, Kuo Cai, Yunfan Wu, Hongtao Cheng, Zexuan Cheng, Lu Ren, Huanjie Wang, Yi Su, Ruiming Tang, Kun Gai, and Guorui Zhou. 2025. OneRec-Think: In-Text Reasoning for Generative Recommendation. arXiv:2510.11639 [cs.IR] <https://arxiv.org/abs/2510.11639>
- [13] Hanjia Lyu, Song Jiang, Hanqing Zeng, Yinglong Xia, Qifan Wang, Si Zhang, Ren Chen, Chris Leung, Jiajie Tang, and Jiebo Luo. 2024. LLM-Rec: Personalized Recommendation via Prompting Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 583–612. doi:10.18653/v1/2024.findings-naacl.39
- [14] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H. Chi. 2018. Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts (*KDD '18*). Association for Computing Machinery, New York, NY, USA, 1930–1939. doi:10.1145/3219819.3220007
- [15] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevech, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Sir Charles Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Mikhail Smelyanskiy. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. *ArXiv abs/1906.00091* (2019). <https://api.semanticscholar.org/CorpusID:173990641>
- [16] Netflix Technology Blog. 2025. Foundation Model for Personalized Recommendation. <https://netflixtechblog.com/foundation-model-for-personalized-recommendation-1a0bd8e02d39>.
- [17] Netflix Technology Blog. 2026. In-House LLM Serving at Netflix. <https://netflixtechblog.com/in-house-llm-serving-at-netflix-a5a8e799ea2c>.
- [18] Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, and Jeremy Hadfield. 2025. Effective context engineering for AI agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>. Anthropic Tech Blog.
- [19] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Keshavan, Trung Vu, Lukasz Heidt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender systems with generative retrieval. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (*NIPS '23*). Curran Associates Inc., Red Hook, NY, USA, Article 452, 17 pages.
- [20] Anima Singh, Trung Vu, Nikhil Mehta, Raghunandan Keshavan, Maheswaran Sathiamoorthy, Yilin Zheng, Lichan Hong, Lukasz Heldt, Li Wei, Devansh Tandon, Ed Chi, and Xinyang Yi. 2024. Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems* (Bari, Italy) (*RecSys '24*). Association for Computing Machinery, New York, NY, USA, 1039–1044. doi:10.1145/3640457.3688190
- [21] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (Beijing, China) (*CIKM '19*). Association for Computing Machinery, New York, NY, USA, 1441–1450. doi:10.1145/3357384.3357895
- [22] Gary Tang, Jiangwei Pan, Henry Wang, and Justin Basilico. 2023. Reward innovation for long-term member satisfaction. In *Proceedings of the 17th ACM Conference on Recommender Systems* (Singapore, Singapore) (*RecSys '23*). Association for Computing Machinery, New York, NY, USA, 396–399. doi:10.1145/3604915.3608873
- [23] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, Hui Xiong, and Enhong Chen. 2024. A Survey on Large Language Models for Recommendation. arXiv:2305.19860 [cs.IR] <https://arxiv.org/abs/2305.19860>
- [24] Zhenrui Yue, Sara Rabhi, Gabriel de Souza Pereira Moreira, Dong Wang, and Even Oldridge. 2023. LlamaRec: Two-Stage Recommendation using Large Language Models for Ranking. arXiv:2311.02089 [cs.IR] <https://arxiv.org/abs/2311.02089>
- [25] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. arXiv:2402.17152 [cs.LG] <https://arxiv.org/abs/2402.17152>
- [26] Changsheng Zhao, Ernie Chang, Zechun Liu, Chia-Jung Chang, Wei Wen, Chen Lai, Sheng Cao, Yuandong Tian, Raghuraman Krishnamoorthi, Yangyang Shi, et al. 2025. Mobilellm-r1: Exploring the limits of sub-billion language model reasoners with open training recipes. *arXiv preprint arXiv:2509.24945* (2025).
- [27] Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, and Ed Chi. 2019. Recommending what video to watch next: a multitask ranking system. In *Proceedings of the 13th ACM conference on recommender systems*. 43–51.
- [28] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.